

```

import sqlite3
import os

from PySide6.QtWidgets import QStyle, QTreeView
from PySide6.QtGui import QStandardItem, QStandardItemModel
from PySide6.QtCore import Signal, Qt, QSortFilterProxyModel

class FileTreeView(QTreeView):
    file_requested = Signal(str)

    def __init__(self, model=QStandardItemModel(), parent=None):
        super().__init__(parent)

        # Disable all editing
        self.setEditTriggers(QTreeView.NoEditTriggers)
        # Disable Drag and Drop
        self.setDragEnabled(False)
        self.setAcceptDrops(False)
        self.setDragDropMode(QTreeView.NoDragDrop) # Prevents internal moves

        self.setModel(model)

        self.setHeaderHidden(True)
        for i in range(1, 4):
            self.setColumnHidden(i, True)

        self.doubleClicked.connect(self._on_double_click)

    def _get_source_model(self):
        """Helper to safely retrieve the underlying QStandardItemModel."""
        curr_model = self.model()
        if isinstance(curr_model, QSortFilterProxyModel):
            return curr_model.sourceModel()
        return curr_model

    def _on_double_click(self, proxy_index):
        """Maps proxy index to source model to retrieve file data."""
        model = self.model()

        # 1. Map index if using a proxy
        if isinstance(model, QSortFilterProxyModel):
            source_index = model.mapToSource(proxy_index)
            source_model = model.sourceModel()
        else:
            source_index = proxy_index
            source_model = model

        # 2. Extract data via itemFromIndex
        if hasattr(source_model, 'itemFromIndex'):
            item = source_model.itemFromIndex(source_index)
            is_dir = item.data(Qt.UserRole)
            path = item.data(Qt.UserRole + 1)

            if not is_dir and path:
                self.file_requested.emit(path)

    def save_to_db(self, db_path):
        """Saves the tree structure safely."""
        # 1. Open connection
        conn = sqlite3.connect(db_path)
        try:
            cursor = conn.cursor()
            # 2. Reset the specific table
            cursor.execute("DROP TABLE IF EXISTS file_tree")
            cursor.execute("""
                CREATE TABLE file_tree (
                    id INTEGER PRIMARY KEY,
                    parent_id INTEGER,
                    name TEXT,
                    path TEXT,
                    is_dir INTEGER
                )""")

            # 3. Define the recursive function inside, using the active cursor
            def walk(parent_item, p_id=None):
                for i in range(parent_item.rowCount()):
                    child = parent_item.child(i)

                    # Pull metadata
                    is_dir = child.data(Qt.UserRole)
                    path = child.data(Qt.UserRole + 1)

                    cursor.execute("""
                        INSERT INTO file_tree (parent_id, name, path, is_dir)
                        VALUES (?, ?, ?, ?)""",
                        (p_id, child.text(), path, int(is_dir) if is_dir is not None else 0))

            walk(self.model())

```

```

        current_id = cursor.lastrowid
        if child.hasChildren():
            walk(child, current_id)

    # 4. Start the walk
    source_model = self._get_source_model()
    walk(source_model.invisibleRootItem())

    # 5. Commit while connection is still open
    conn.commit()

except sqlite3.Error as e:
    print(f"Database error during save: {e}")
finally:
    # 6. Always close in the 'finally' block
    conn.close()

def load_from_db(self, db_path):
    """Reconstructs the file tree from the SQLite database."""
    if not os.path.exists(db_path):
        return

    # 1. Access the correct source model (StandardItemModel)
    source_model = self._get_source_model()
    source_model.clear()

    try:
        # 2. Connect and fetch rows using Row factory for name-based access
        conn = sqlite3.connect(db_path)
        conn.row_factory = sqlite3.Row
        cursor = conn.cursor()

        # Ensure we are targeting the 'file_tree' table
        rows = cursor.execute("SELECT id, parent_id, name, path, is_dir FROM file_tree").fetchall()
        conn.close()
    except sqlite3.OperationalError as e:
        print(f"Database Table not found or error: {e}")
        return

    items = {}

    # Pass 1: Create all QStandardItem and store metadata
    for r in rows:
        item = QStandardItem(r['name'])

        # Access columns using brackets [ ] instead of .get()
        path = r['path']
        # Convert integer (0/1) from SQLite back to Boolean
        is_dir = bool(r['is_dir'])

        # Restore the custom UserRoles for app functionality
        item.setData(is_dir, Qt.UserRole)    # Role 0: is_dir
        item.setData(path, Qt.UserRole + 1)  # Role 1: full path

        # Restore icons based on folder/file status
        icon_type = QStyle.SP_DirIcon if is_dir else QStyle.SP_FileIcon
        item.setIcon(self.style().standardIcon(icon_type))

        # Store in dictionary by its database ID for the next pass
        items[r['id']] = item

    # Pass 2: Build the parent-child hierarchy
    for r in rows:
        item = items[r['id']]
        p_id = r['parent_id']

        if p_id and p_id in items:
            # Add this item as a child to its parent
            items[p_id].appendRow(item)
        else:
            # If no parent_id, add it as a top-level root item
            source_model.invisibleRootItem().appendRow(item)

    # Auto-expand the restored tree
    self.expandAll()

def toggle_visibility(self):
    self.setVisible(not self.isVisible())

```